RLinf-VLA

视觉语言动作模型的强化学习原理与实现

多粒度策略优化、Action-level Critic 与训练系统

Yang kunpeng

论文 arxiv.org/abs/2510.06710

项目主页 rlinf-vla.github.io

官方代码 github.com/RLinf/RLinf

目录

第一章 动作结构与在线训练系统 **1**

1.1 Action Chunk 的层级结构与策略概率 1

1.2 Rollout 数据及统一接口 3

1.3 Generation、Simulator 与 Training 的协同执行 5

第二章 多粒度 PPO、GRPO 与参数更新 **7**

2.1 Action-level Critic、GAE 与 PPO 7

2.2 GRPO 的轨迹奖励、Mask 与长度归一化 11

2.3 参数更新与训练闭环 13

参考资料 **16**

第 1 章

动作结构与在线训练系统

1.1 Action Chunk 的层级结构与策略概率

视觉语言动作模型接收视觉观测、语言目标及可选的机器人本体状态，输出供控制器执行的动作。对于一次预测多个控制时间步的策略，机器人动作必须同时区分**动作块（Action Chunk）**、**原子动作（Atomic Action）**和**动作 Token**。这三个层次对应不同的环境时间尺度和概率计算单位。

设第 t 次模型推理时的观测为 o_t，动作块长度为 C，原子动作维数为 M。定义

$$c_t = (a_{t,1}, \ldots, a_{t,C}), \qquad a_{t,i} = (d_{t,i,1}, \ldots, d_{t,i,M}). \tag{1.1}$$

其中 t 是**动作块索引**，i 是块内原子动作索引，j 是动作维度或对应的离散 Token 索引。每个 $a_{t,i}$ 由机器人控制器在一个原子控制时间步执行。模型在 o_t 上生成整个 c_t，环境随后执行这个块并返回下一次观测，因而 VLA 在**块与块之间**闭环，在一次块执行期间不必重新调用完整的 VLA 策略。

以七维末端执行器控制为例，$a_{t,i} \in \mathbb{R}^7$ 可包含三个平移分量、三个旋转分量和夹爪控制量。若 $C = 8$，连续控制数组的形状为 $[8, 7]$。当动作头采用离散动作编码时，可以对应到 8×7 个离散 Token。这个形状只是**分层动作表示**的实例；对于连续回归头，不能把七个连续数值自动视为具有分类 Softmax 的 Token。

三种概率计算粒度

PPO 和 GRPO 都需要旧策略与当前策略在同一已采样动作上的概率或密度。论文在离散动作 Token 的设定下，将概率计算粒度统一为：

$$\text{Token-level:} \quad p_{t,i,j} = \pi_\theta(d_{t,i,j} \mid o_t, d_{t,i,:j-1}), \tag{1.2}$$

$$\text{Action-level:} \quad \pi_\theta(a_{t,i} \mid o_t, a_{t,:i-1}) = \prod_{j=1}^{M} p_{t,i,j}, \tag{1.3}$$

$$\text{Chunk-level:}\quad \pi_\theta(c_t \mid o_t) = \prod_{i=1}^{C} \pi_\theta(a_{t,i} \mid o_t, a_{t,:i-1}). \tag{1.4}$$

这里的乘积分解表示模型使用的条件概率结构；具体条件信息仍须与真实动作解码器匹配。尤其是并行预测的策略，不应仅凭上述记号断言其在机器人控制维度上采用逐 Token 自回归生成。

实际实现使用对数域求和避免直接相乘。记 $\ell_{t,i,j} = \log p_{t,i,j}$，则

$$\log \pi_\theta(a_{t,i} \mid \cdot) = \sum_{j=1}^{M} \ell_{t,i,j}, \qquad \log \pi_\theta(c_t \mid o_t) = \sum_{i=1}^{C} \sum_{j=1}^{M} \ell_{t,i,j}. \tag{1.5}$$

因此，假定原始 Log Probability 张量形状为 $[B, C, M]$，其中 B 是参与当前计算的样本数，则不同粒度的数据形状分别为

计算方式	计算操作	输出形状
Token-level	保留每个 Token 的 ℓ	$[B, C, M]$
Action-level	沿动作维度 M 求和	$[B, C]$
Chunk-level	沿 C, M 求和	$[B]$

这些粒度会改变优化问题，而不只是改变张量形状。设 $\rho_j = \exp(\ell_{\theta,j} - \ell_{\text{old},j})$，则 Chunk-level 的重要性采样比值为 $\prod_j \rho_j$，Token-level 是对每个 ρ_j 分别裁剪。一般有

$$\text{clip}\left(\prod_j \rho_j, 1-\epsilon, 1+\epsilon\right) \neq \prod_j \text{clip}(\rho_j, 1-\epsilon, 1+\epsilon). \tag{1.6}$$

例如八个原子动作各自的比值均为 1.1，则整个动作块的联合比值为 $1.1^8 \approx 2.14$；它可能已经触及 Chunk-level 裁剪上界，而每个 Action-level 比值仍在区间内。概率粒度因而直接影响损失函数的裁剪位置和梯度。

概率粒度与 Advantage 粒度的组合

优势估计的粒度和概率计算粒度不是同一概念。论文支持两种 Advantage 粒度：一个块共享一项优势，或者一个块内的 C 个动作分别拥有优势。原文给出的兼容性如下：

Advantage	Chunk 概率	Action 概率	Token 概率
Chunk-level	支持	支持	支持
Action-level	不支持	支持	支持

Action-level Advantage 不能直接与单个 Chunk-level Ratio 按位置对应，因此框架不支持这一组合。Action-level Advantage 配合 Token-level Ratio 时，一个原子动作的优势可以广播到构成该动作的各个 Token，但这些 Token 并未因此获得独立的环境奖励。

官方代码中的粒度变换

官方仓库 `rlinf/algorithms/utils.py` 的 `preprocess_loss_inputs` 会将已经计算出的 Log Probability 按 `logprob_type` 变换。以下以相同数据操作写成等价的紧凑表达，说明其维度含义：

```
# logp, old_logp: [B, C*M]
new = logp.reshape(B, C, M)
old = old_logp.reshape(B, C, M)
if level == "action_level":
    new, old = new.sum(-1), old.sum(-1)  # [B,C]
elif level == "chunk_level":
    new, old = new.sum((1, 2)), old.sum((1, 2))  # [B]
ratio = (new - old).exp()
```

在粒度变换后，算法还会同步调整 Advantage、Value、Return 和 Loss Mask 的形状，使它们能够与选择的策略概率粒度广播对齐。官方代码入口：`https://github.com/RLinf/RLinf/blob/main/rlinf/algorithms/utils.py`。

1.2 Rollout 数据及统一接口

一条 VLA 在线强化学习轨迹由模型和环境交替产生。令 o_t 是第 t 个块开始时的观测，c_t 是策略采样得到的动作块，环境执行后产生逐原子动作奖励 $r_{t,1}, \ldots, r_{t,C}$、终止标记和下一块观测 o_{t+1}。其基本迭代为

$$o_t \xrightarrow{\pi_{\theta_{\text{old}}}} c_t \xrightarrow{\text{Simulator}} (r_{t,1:C}, o_{t+1}, \text{done}_t). \tag{1.7}$$

这里的 o_{t+1} 表示**下一次动作块推理的观测**，并非块内第一个原子动作后的观测。对 Chunk 内每个原子动作计算价值时，环境虽然逐动作推进，但策略网络未必获得逐动作的图像观测。

为了使旧数据可用于 PPO/GRPO 更新，每个样本至少保留已执行动作（或离散动作 Token）、对应观测、旧策略 Log Probability、环境奖励和终止信息。若使用 PPO，还要保存旧 Critic 的 Value 预测或在训练前计算对应 Value；如果使用有效时间步筛选，则需要与动作时间轴对齐的 Mask。

批量张量的时间组织

设一次 Rollout 收集 K 个 Chunk，每个 Chunk 有 C 个原子动作，并行环境数为 B。典型张量形状为

数据	结构
观测	$[K, B, \ldots]$，每个 Chunk 一次输入
原子奖励	$[K, B, C]$
终止标记	$[K, B, C]$ 或带边界项的形式
Action-level Value	$[K+1, B, C]$（含引导值）
Token Log Probability	$[K, B, C, M]$（概念维度）
Action-level Advantage	$[K, B, C]$
Chunk-level Advantage	$[K, B, 1]$

上表中的形状是理解时间轴和动作轴的数学记法，实际数据容器可将前两维展平或按训练框架重新排列。PPO 最重要的运算是把 Action-level 奖励和价值按控制时间先后变为一个长度为 KC 的时间序列，而不是将不同维度误当作互相独立的环境。

官方函数 `preprocess_embodied_advantages_inputs` 对 Chunk-level 奖励使用最后一维求和，对 Chunk 内终止状态进行汇聚；Action-level 分支则保留原子动作的时间分辨率，再经过轴交换和展平，将 $[K, B, C]$ 映射为 $[KC, B]$。这一步发生在 GAE 和 GRPO 等 Advantage 计算之前。对应的官方逻辑可归纳为：

```
# reward: [K, B, C]
if reward_type == "chunk_level":
    reward = reward.sum(-1, keepdim=True)
# 将块内时间接到块间时间之后
```

```
T = reward.shape[0] * reward.shape[-1]
reward = reward.transpose(1, 2).reshape(T, B)
```

Chunk-level 的时间间隔为一个块，Action-level 的时间间隔为一个原子动作。因此，折扣因子 γ 的含义必须与采用的时间分辨率一致。如果每个控制步折扣为 γ_a，长度固定为 C 的宏动作在处理逐控制步折扣时，跨块引导项相应包含 γ_a^C；简单相加即时奖励并把 γ_a 不加调整地用于块级 TD，并不与动作级回报严格等价。论文支持两种计算粒度，但并未声称二者在任意折扣和奖励约定下数值等价。

统一的 Generation–Simulator–Training 接口

框架将在线训练划分为三个逻辑角色：Generation 负责根据观测生成动作块与概率；Simulator 负责逐控制步执行动作并返回新观测、奖励和终止信息；Training 负责计算 Advantage、策略损失、价值损失并更新参数。一次策略更新完成后，新参数被用于后续 Rollout。

模型适配器处理不同 VLA 的输入、动作解码和动作分布接口；环境适配器规范不同模拟器的 `reset/step` 行为、动作块执行及终止规则。算法只依赖轨迹数据及可配置的 Advantage、Loss，而不把 ManiSkill、LIBERO 或 RoboTwin 的物理控制细节写进 PPO/GRPO 的数学实现。

该接口设计的技术价值，在于把**模型输出概率的定义**与**奖励来源**解耦：更换环境不会改变 PPO 的裁剪公式，而更换动作头会改变 Log Probability 的实现，必须仍然保持同一个动作分布语义。

1.3 Generation、Simulator 与 Training 的协同执行

在线训练的耗时来自三类不同负载：模型前向推理、环境物理仿真与渲染、模型反向传播。这些负载对 GPU 的占用形式并不一致。若全部组件固定复用同一组 GPU，会产生模型切换与显存占用问题；若完全分开，也可能在模型等待观测或模拟器等待动作时出现空闲。

RLinf-VLA 提供三种资源分配方式。**Collocated** 让不同组件共享 GPU 资源，在不同执行阶段切换；**Disaggregated** 将推理和仿真分配到不同 GPU；**Hybrid** 则把推理和环境实例分成细粒度流水线阶段，以交错处理不同子批次。其核心不是改变奖励或 PPO 更新公式，而是减少 Rollout 时各组件相互等待。

当仿真环境可以大规模使用 GPU 并行时，Generation 与 Simulator 可以在不同资源池上交替处理两个或多个子批次。当仿真器主要受 CPU 执行限制时，分离 GPU 并不一定提升吞吐，因为两类设备之间的数据传递与等待仍可能成为瓶颈。框架因此允许根据环境类型配置资源模式，并提供基于运行时测量的自动放置选择。

对学习强化学习算法而言，系统部分只需保留一项约束：**无论如何并行调度，同一条轨迹的动作、旧策略 Log Probability、Reward 和环境终止边界必须保持一致**。如果 Rollout 使用的策略版本与存储概率不匹配，Importance Ratio 就失去原本的统计含义。训练前必须确保所有与旧策略相关的数据拥有明确版本或一致的概率基准。

第 2 章

多粒度 PPO、GRPO 与参数更新

2.1 Action-level Critic、GAE 与 PPO

PPO 通常使用 Actor–Critic：Actor 是根据观测生成动作的 VLA 策略 π_θ；Critic 是预测未来折扣回报的价值函数 V_ϕ。价值函数的目标不是判断某一动作在物理意义上是否正确，而是估计从相应决策位置开始执行当前策略的期望累计奖励。

对于控制时间索引 k，价值函数满足

$$V^\pi(s_k) = \mathbb{E}_\pi\left[\sum_{h=0}^{T-k-1} \gamma^h r_{k+h} \middle| s_k\right]. \tag{2.1}$$

当算法只能获得图像等部分观测时，实际 Critic 输入通常是观测特征与任务条件，其预测可写为 $V_\phi(o_k)$。对 Action Chunk 来说，重要问题是 Critic 在怎样的时间粒度上输出 Value。

单值 Critic 与多值 Critic

Chunk-level Critic 在一次 VLA 推理处预测一个标量：

$$V_\phi(o_t) \in \mathbb{R}. \tag{2.2}$$

整个 Chunk 被视为一个宏动作，Value 与奖励按照块时间轴更新。

Action-level Critic 则在相同的 Chunk 起点观测上一次输出 C 个值：

$$V_\phi(o_t) = \big(V_{t,1}, V_{t,2}, \ldots, V_{t,C}\big) \in \mathbb{R}^C. \tag{2.3}$$

这里的 $V_{t,i}$ 对应块内第 i 个原子动作之前的**价值估计位置**。由于模型在该块执行过程中未必重新观测环境，这些估计来自同一个块起始特征；不能把它误写成 Critic 在每个原子动作之前都取得了新的摄像头图像。

论文采用 Actor 和 Critic 共享模型主干、再添加轻量 Value Head 的方式降低开销。这种共享 Backbone 的思路继承了已有工作，并非 PPO 自身的新算法；RLinf-VLA 在动作块情境下支持输出 $\mathbb{R}$ 或 $\mathbb{R}^C$，并对各自的优势估计进行系统比较。Value Head 的输出维度改变，决定能否获得块内不同动作位置的 Value Target。

GAE 如何从奖励得到不同的 Advantage

把多个 Action Chunk 按执行顺序展开为原子动作时间序列，统一以 k 表示时间。使用从当前 Rollout 策略评估阶段得到的旧 Critic 预测 v_k，定义 TD 残差

$$\delta_k = r_k + \gamma(1 - d_{k+1})v_{k+1} - v_k, \tag{2.4}$$

其中 $d_{k+1} = 1$ 表示在动作 k 后发生真实终止；对于非终止的采样边界，需要根据具体 Bootstrap 规则使用后继状态的 Value，而不是强制置零。

Generalized Advantage Estimation（GAE）为

$$\hat{A}_k = \delta_k + \gamma\lambda(1 - d_{k+1})\hat{A}_{k+1}, \tag{2.5}$$

$$= \sum_{h\geq 0}(\gamma\lambda)^h \Big(\prod_{j=1}^{h}(1 - d_{k+j}) \Big)\delta_{k+h}. \tag{2.6}$$

其中 $\gamma \in [0, 1]$ 为奖励折扣因子，$\lambda \in [0, 1]$ 为 GAE 的混合参数。反向递推时，在真实终止边界或当前采样片段的合适 Bootstrap 边界停止。

Critic 的训练目标可以用旧价值预测与 GAE 相加得到：

$$\hat{G}_k = \text{stopgrad}(\hat{A}_k + v_k). \tag{2.7}$$

Critic 使用 $\hat{G}_k$ 作为固定目标。基础的均方误差形式为

$$L_V(\phi) = \frac{1}{N}\sum_{k=1}^{N}\big(V_\phi(o_{\text{chunk}(k)}, i(k)) - \hat{G}_k\big)^2. \tag{2.8}$$

这里 chunk(k) 是原子动作 k 所属的 Chunk，$i(k)$ 是该动作在 Chunk 内的位置。

RLinf 官方 `compute_ppo_critic_loss` 实际还使用 Value Clipping 与 Huber 损失。令 v_k^{old} 为更新前预测，v_k 为当前预测，ϵ_v 为价值裁剪宽度，则

$$\tilde{v}_k = v_k^{\text{old}} + \text{clip}(v_k - v_k^{\text{old}}, -\epsilon_v, \epsilon_v), \tag{2.9}$$

$$L_V = \text{Mean}_{m_k=1}\Big[\max\big(H_\delta(\hat{G}_k - v_k), H_\delta(\hat{G}_k - \tilde{v}_k)\big)\Big]. \tag{2.10}$$

H_δ 为阈值为 δ 的 Huber 损失，m_k 是有效位置 Mask。取未裁剪与裁剪预测对应损失的较大值，可避免 Critic 通过过大的单步变化获得不合理的优化收益。

三个动作的完整数值计算

设一个 Chunk 内有三个连续执行的原子动作，最终一步任务成功，奖励为

$$r = (0, 0, 1), \qquad v = (0.2, 0.3, 0.6), \qquad v_3 = 0. \tag{2.11}$$

取 $\gamma = 0.9$、$\lambda = 0.95$。末尾是实际终止状态，$v_3 = 0$。三个 TD 残差为

$$\delta_0 = 0 + 0.9 \times 0.3 - 0.2 = 0.07, \tag{2.12}$$

$$\delta_1 = 0 + 0.9 \times 0.6 - 0.3 = 0.24, \tag{2.13}$$

$$\delta_2 = 1 - 0.6 = 0.40. \tag{2.14}$$

从末尾向前计算：

$$\hat{A}_2 = 0.40, \tag{2.15}$$

$$\hat{A}_1 = 0.24 + 0.9 \times 0.95 \times 0.40 = 0.582, \tag{2.16}$$

$$\hat{A}_0 = 0.07 + 0.9 \times 0.95 \times 0.582 \approx 0.568. \tag{2.17}$$

由此可得

$$(\hat{A}_0, \hat{A}_1, \hat{A}_2) \approx (0.568, 0.582, 0.400), \tag{2.18}$$

以及价值监督目标

$$(\hat{G}_0, \hat{G}_1, \hat{G}_2) \approx (0.768, 0.882, 1.000). \tag{2.19}$$

这三个 Advantage 不同，原因不是环境为三个动作提供了完全不同的最终成功判定，而是三个动作位置具有不同的价值预测和 TD 残差。同一个成功回报也可以产生不同的相对预期改善量。更大的 Advantage 并不必然表示那个动作物理上更重要，它只反映当前价值基线下的优化信号。

如果改用 Chunk-level Critic，则只有一个块级 Value，无法在该 Chunk 内直接计算三组这样的 TD 残差与 GAE。对块级方法而言，每个 Chunk 的奖励及跨块 Bootstrap 构成其时间序列。因此两种方案的区别不仅是 Value Head 最后一层从 1 改成 C，还包括**价值目标的时间轴**。

官方输入预处理与 GAE 的对应

官方 `preprocess_embodied_advantages_inputs` 在 `reward_type=action_level` 时保留 C 个原子奖励，再把 $[K, B, C]$ 经维度交换展平为 $[KC, B]$，从而使 GAE 按原子动作时序递推；随后 `postprocess_embodied_advantages_outputs` 把优势还原为 $[K, B, C]$，以便与 Chunk 数据和 Loss 位置对齐。

这种预处理还要同步处理 done 与 Bootstrap Value 的长度，使 v_{k+1} 确实指向执行顺序里的下一个价值位置，而不是同一维度上的另一条并行环境。它是实现 Action-level Critic 的必要条件。

PPO 策略损失

收集 Rollout 的旧策略记为 $\pi_{\theta_{\text{old}}}$。对已采样的动作单位 u_k，当前策略重新计算

$$\rho_k(\theta) = \exp\left(\log \pi_\theta(u_k \mid o_k) - \log \pi_{\theta_{\text{old}}}(u_k \mid o_k)\right). \tag{2.20}$$

其中 u_k 的含义必须与上一章选择的 Action/Token/Chunk 概率粒度一致。PPO 的裁剪策略目标为

$$J_{\text{PPO}}(\theta) = \mathbb{E}_k\left[\min\left(\rho_k \hat{A}_k, \text{clip}(\rho_k, 1-\epsilon, 1+\epsilon)\hat{A}_k\right)\right]. \tag{2.21}$$

Actor 最大化 J_{PPO}，等价于最小化 $-J_{\text{PPO}}$。Critic 另外最小化 L_V。策略损失的梯度从已采样动作的 Log Probability 回传到可训练的 VLA 参数，并不需要让梯度穿过机器人仿真器。

官方 `losses.py` 将 Actor 和 Critic 损失分别计算。其主要张量运算为：

```
ratio = (new_logp - old_logp).exp()
clipped = ratio.clamp(1.0 - eps_low, 1.0 + eps_high)
actor_terms = torch.maximum(-adv * ratio, -adv * clipped)
actor_loss = masked_mean(actor_terms, valid_mask)

v_clip = old_value + (value - old_value).clamp(-eps_v, eps_v)
critic_terms = torch.maximum(huber(target - value),
                             huber(target - v_clip))
critic_loss = masked_mean(critic_terms, valid_mask)
```

这里使用与官方 Actor、Critic 损失一致的主要张量算子；实际函数还包含额外的比值稳定化、Dual Clip、训练指标和损失聚合选项。以上各项以配置为准。官方函数入口：`https://github.com/RLinf/RLinf/blob/main/rlinf/algorithms/losses.py`。

对于 Action-level Advantage 配合 Token-level Log Probability 的设定，同一动作的 $\hat{A}_{t,i}$ 可被分配给它所包含的每一个 Token；各 Token 仍具有各自的 $\rho_{t,i,j}$。对于 Action-level 概率则先把相应 Token Log Probability 求和，再计算一次 $\rho_{t,i}$。这两种策略损失的裁剪位置不同，即使使用同样的优势值也并非完全等价。

2.2 GRPO 的轨迹奖励、Mask 与长度归一化

GRPO 不训练单独的 Critic，而是从同一任务条件下采样 G 条轨迹，用组内回报差异估计优势。设 R_i 为第 i 条轨迹的总奖励，则标准化的组相对优势为

$$\hat{A}_i = \frac{R_i - \bar{R}}{\mathrm{std}(R_1, \ldots, R_G) + \varepsilon}, \qquad \bar{R} = \frac{1}{G}\sum_{j=1}^{G} R_j. \tag{2.22}$$

这里的 $\hat{A}_i$ 是**轨迹级**标量；给该轨迹的多个动作或 Token 计算损失时，通常共享这一优势。GRPO 不会因为动作块中有多个原子动作，就自动产生 PPO 中基于 Value 的逐动作 Advantage。

无组内奖励差异的情形

对二值结果奖励，若一组中全部失败，则 $R_i = 0$；若全部成功，则 $R_i = 1$。这两种情况下中心化的优势均为零。Success Rate Filter 因此筛选混合成功和失败结果的有效组，以免在组内比较中使用大量没有相对奖励信号的数据。

这个过滤器不改变 GRPO 的基本公式，也不能凭空生成成功经验。如果当前策略无法采样到任何成功轨迹，就可能继续面临探索不足和梯度信号缺失的问题。论文的实验还表明，过滤器的收益具有模型与任务依赖性，并非始终优于不筛选。

Valid Action Mask

当 Rollout 采用固定交互长度，而某一任务在中途已完成时，任务成功后的动作可能与优化的成功条件无关。定义有效动作指示变量

$$m_{i,k} = \begin{cases} 1, & \text{第} i \text{ 条轨迹中的时间步} k \text{ 参与优化,} \\ 0, & \text{该时间步被排除.} \end{cases} \tag{2.23}$$

Mask 表示**损失是否使用这个位置**，不能与环境终止标记 d_k 混为一谈。终止标记控制价值引导和轨迹边界；Mask 控制哪些采样位置进入最终损失。在论文的 GRPO 训练中，Valid Action Mask 用于排除任务成功后的无关动作，而 PPO 的 Partial Reset 则是在环境采样阶段立即重置已终止的子环境。这两者作用于不同流程。

按有效轨迹长度归一化

将有效时间步数定义为 $T_i^{\text{valid}} = \sum_k m_{i,k}$。对于第 i 条轨迹，每个有效位置的裁剪项记为

$$f_{i,k} = \min\left(\rho_{i,k}(\theta)\hat{A}_i, \text{clip}(\rho_{i,k}(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_i\right). \tag{2.24}$$

论文附录给出的长度归一化 GRPO 目标写为

$$J_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{T_i^{\text{valid}}}\sum_{k=1}^{T_i^{\text{valid}}} f_{i,k}\right]. \tag{2.25}$$

与无长度归一化的求和式相比，每条轨迹先对自身的有效步数取平均，再进行组平均，能够防止长轨迹仅因时间步更多而主导整体梯度。当概率选择 Token-level 时，还必须明确 k 的统计单位是原子动作、Token，还是经过 Mask 映射后的训练位置；不能在不同统计单位之间混用同一个长度分母。

损失权重的数值区别

假定两条轨迹均具有固定的组内优势，第一条包含 20 个有效位置，第二条包含 100 个有效位置。若直接在全体位置上取一个平均，第一条轨迹只占 $20/(20+100) = 1/6$ 的位置权重。若按照论文的先轨迹内平均、再组平均，两条轨迹各占 $1/2$ 的组权重。二者所依赖的轨迹奖励可以完全相同，但优化目标对长短轨迹的相对侧重不同。

长度归一化并非通用奖励塑形，也没有直接使机器人更偏好更短的成功路径；它首先改变的是**训练损失中不同轨迹的聚合权重**。论文在 LIBERO-Goal 的相关设置中观察到收益，但在 ManiSkill 中并未看到同样清晰的改善，因此应根据任务终止语义与 Rollout 采样策略选择。

GRPO 与 PPO 的本质分工

	PPO	GRPO
优势来源	TD 残差与 GAE	组内轨迹回报比较
Critic	通常需要	无独立 Critic
块内信用分配	可逐原子动作	基础版本共享轨迹级优势
主要问题	Value 学习和引导边界	稀疏奖励与有效轨迹组

PPO 与 GRPO 都可能按 Token、原子动作或 Chunk 计算概率比值。区别首先是**怎样获得 Advantage**，其次才是将 Advantage 与哪一种粒度的 Ratio 对齐。

2.3 参数更新与训练闭环

一次完整的在线更新可按下列计算依赖组织：

1. 当前策略在多个机器人环境中生成 Action Chunk，并保存采样动作与旧策略概率。
2. Simulator 依次执行块内原子动作，形成奖励、终止标记、下一个块观测及有效动作信息。
3. 根据 `reward_type` 将奖励与 Value 映射到 Chunk-level 或 Action-level 时间轴。
4. PPO 使用 GAE 计算优势和 Critic 的回报目标；GRPO 使用组内轨迹回报计算相对优势。
5. 根据 `logprob_type` 整理新旧 Log Probability，并把 Advantage、Mask 与 Ratio 对齐。
6. 对有效位置计算裁剪策略损失；PPO 额外优化 Value Loss。对模型可训练参数反向传播。
7. 同步新策略至 Generation，进入下一次 Rollout。

这条链路中，reward_type 控制价值/奖励时间轴，logprob_type 控制策略比值粒度，adv_type 选择 GAE 或 GRPO 等优势估计方法。官方配置还提供 γ、λ、裁剪阈值、Value Clipping、Bootstrap 类型与损失聚合方式，具体组合必须满足上一章的粒度兼容性。

逐动作价值目标与策略损失的数据连接

设一个 Chunk 长度为 $C = 3$，某次 GAE 得到

$$(\hat{A}_1, \hat{A}_2, \hat{A}_3) = (0.568, 0.582, 0.400). \tag{2.26}$$

若其 Action-level Ratio 为 $(1.10, 0.92, 1.30)$，设上侧裁剪阈值为 1.2、下侧为 0.8，则正优势的逐动作裁剪目标分别为

$$f_1 = 1.10 \times 0.568 = 0.6248, \tag{2.27}$$

$$f_2 = 0.92 \times 0.582 = 0.53544, \tag{2.28}$$

$$f_3 = \min(1.30, 1.20) \times 0.400 = 0.4800. \tag{2.29}$$

第三个原子动作的比值已超过上界，它的正优势目标不再随该比值增加。若将整个 Chunk 的概率先相乘，则联合比值为 $1.10 \times 0.92 \times 1.30 = 1.3156$，而整个 Chunk 需要匹配一个块级 Advantage；这既不是上面三项的平均，也不能把三个 Action-level Advantage 随意塞入一个标量裁剪项。

反向传播从策略 Log Probability 开始。对未触发裁剪平台的某个动作单位，在忽略聚合系数时，负目标对 Log Probability 的梯度为

$$\frac{\partial L_\pi}{\partial \log \pi_\theta(u_k \mid o_k)} = -\rho_k \hat{A}_k. \tag{2.30}$$

正优势倾向于提高该动作单位的概率；负优势倾向于降低其概率。触发相应的平坦裁剪分支时，这一方向上的局部梯度为零。若 Actor 与 Critic 共享主干，策略损失及 Value Loss 都可能对共享参数产生梯度；哪些参数参与训练，受冻结配置及 LoRA/全参数训练策略控制。

在线训练与模型推理的差别

训练需要随机采样、环境反馈、旧策略概率、Advantage、Mask 和参数更新；部署推理仅需要根据当前观测生成动作块，将其交给机器人执行，再获取下一块观测。

Critic 在 PPO 训练时提供优势和回报目标，但控制机器人的是 Actor；GRPO 训练使用的轨迹组比较也不是部署时的必要环节。

RLinf-VLA 的统一训练框架同时服务不同动作编码方式，但**策略概率是模型相关量**。对离散 Action Token 模型可使用分类概率；若模型使用其他连续动作生成机制，就必须由对应的概率化策略或采样方法提供合法的 Log Probability。不能把一个仅输出确定性动作的回归头直接代入 PPO/GRPO 的概率比值公式。

算法边界

Action-level Critic 通过 C 个 Value Target 提供了比 Chunk-level 更细的时间信用分配，但它并不能保证准确定位导致任务失败的原子动作。因为块内 Value 可能由同一次起始观测推断，缺少逐动作的新图像反馈，且目标本身受奖励稀疏性、价值估计偏差和 Bootstrap 误差影响。

另一方面，使用 Chunk-level 的宏动作抽象可减少价值预测数目，但会牺牲时间分辨率；将多个 Token 的 Ratio 相乘则更容易让联合比值越过裁剪边界。**选择粒度实际上是在价值估计误差、信用分配时间分辨率与策略更新稳定性之间进行权衡**。论文的算法贡献在于提供并验证这些可配置计算方式，同时通过统一接口和细粒度调度，使多模型、多环境的在线强化学习能够在一个系统中实现。

参考资料

1. Zang, H. et al. *RLinf-VLA: A Unified and Efficient Framework for Reinforcement Learning of Vision-Language-Action Models.* arXiv:2510.06710, 2026. https://arxiv.org/abs/2510.06710
2. RLinf Authors. *RLinf*: 官方代码仓库。https://github.com/RLinf/RLinf
3. RLinf Authors. *Embodiment Configuration*: 官方算法与模型配置接口。https://rlinf.readthedocs.io/en/latest/rst_source/guides/embodiment_config.html
4. Schulman, J. et al. *Proximal Policy Optimization Algorithms.* arXiv:1707.06347, 2017.
5. Schulman, J. et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation.* arXiv:1506.02438, 2015.